

EE488 HW3 Report

학번 비공개 Sim Hyeonseong

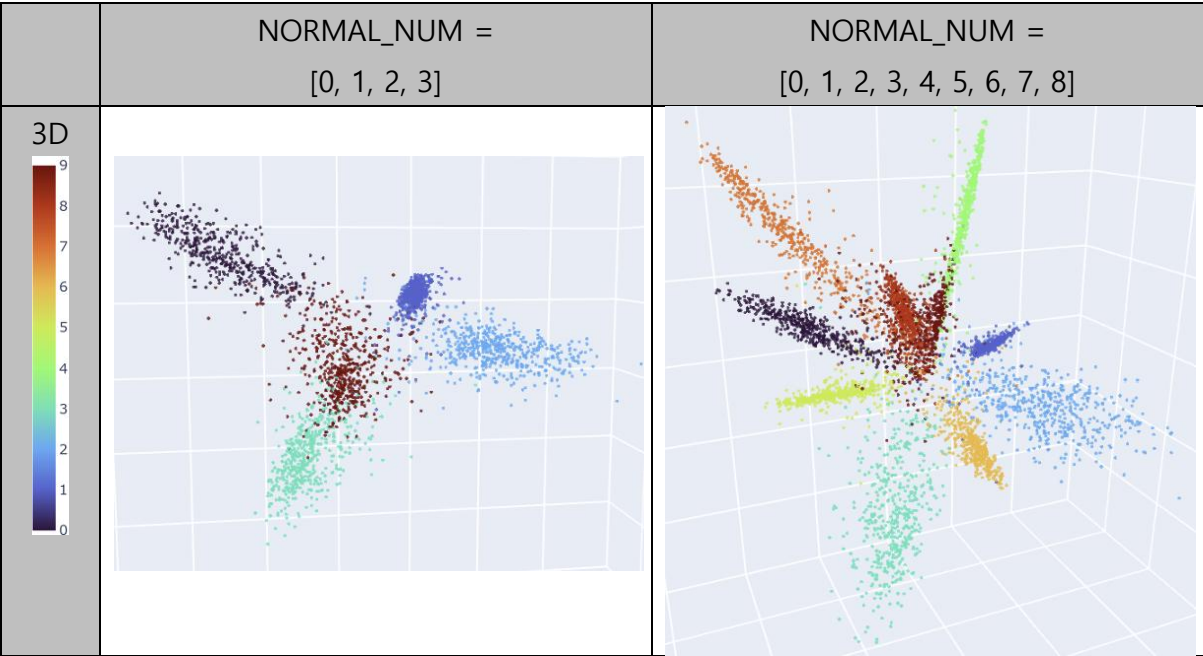
[Problem 1]

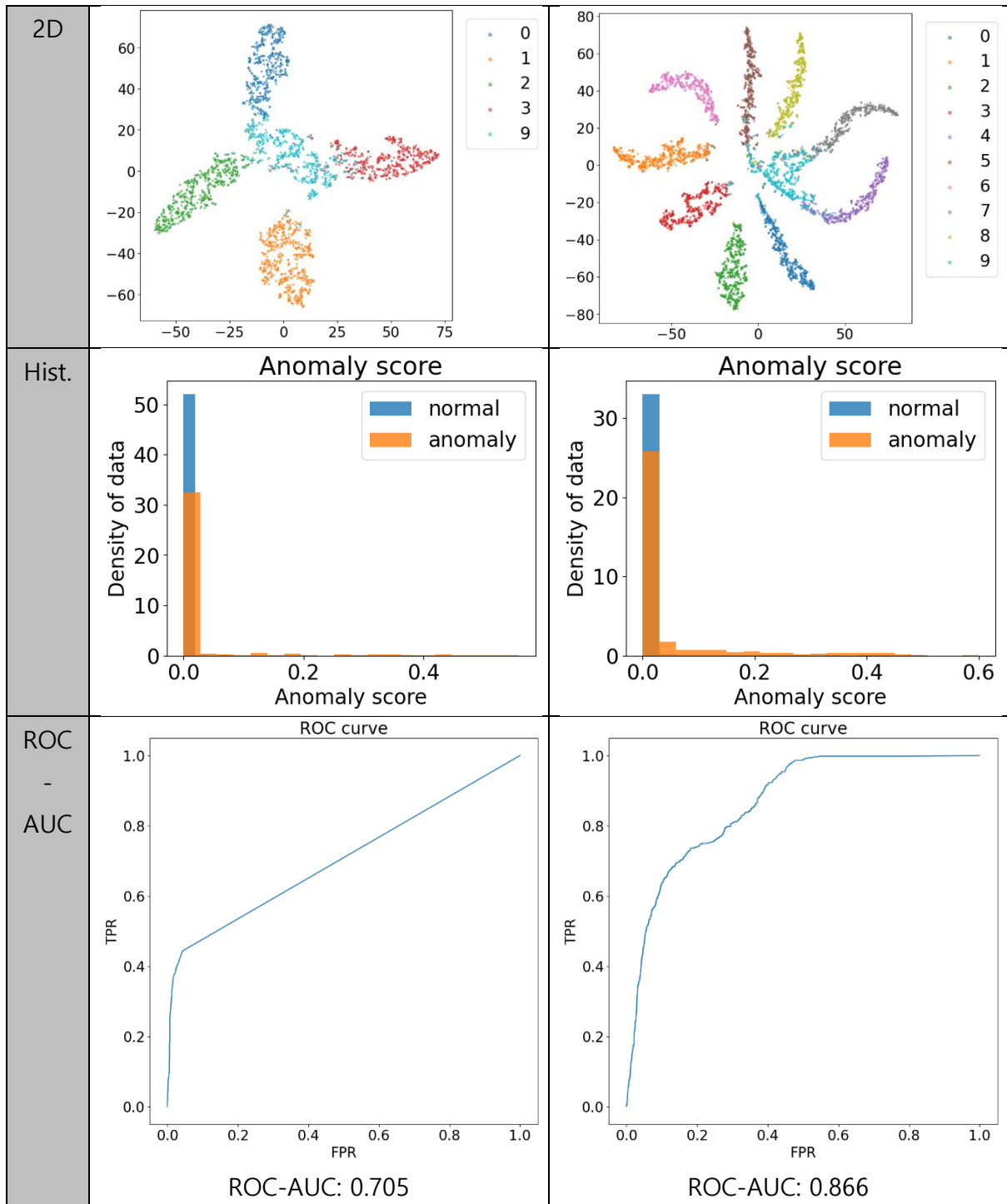
```
import random

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
torch.cuda.manual_seed(42)
```

```
ANOMALY_NUM = [9] # (list) Digits will be used as anomalous data
# NORMAL_NUM = [0,1,2,3] # (list) Digits used as normal data
NORMAL_NUM = [0,1,2,3,4,5,6,7,8] # (list) Digits used as normal data
```

At the beginning of the skeleton code, I added some lines to fix random seeds for reproducibility. I trained the model for two cases with smaller number of normal classes and larger one. The chart below shows the scatter plots in 3D and 2D, anomaly score histograms, and ROC-AUCs with ROC curves.





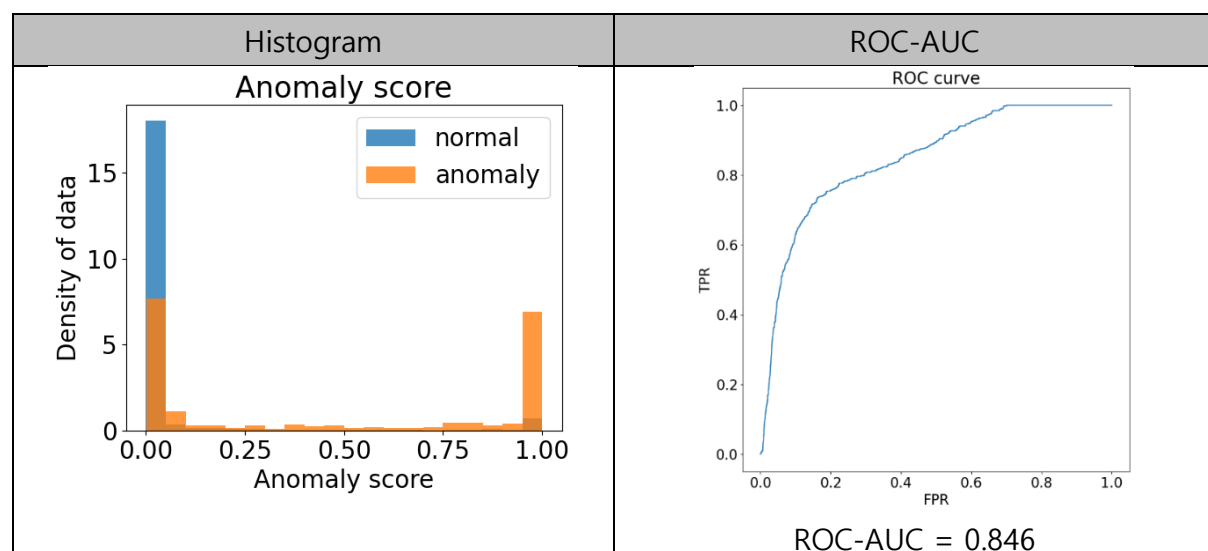
According to the results above, the model trained with larger number of normal classes shows better performance (higher ROC-AUC). Comparing scatter plots both in 3D and 2D, the average intra-class distances within each normal class are smaller, and the intra-class distribution looks a little bit sharper and more compact in more normal classes. The scatter plots are drawn considering the magnitudes of feature vectors. But when distinguishing whether the given input is anomaly or not, the model just checks the angles between the feature vector and the weight vectors. Since feature vectors of 9 (anomalies) are randomly

but quite uniformly distributed toward all directions from the center when NORMAL_NUM = [0, 1, 2, 3], it results in higher probability of small angles between the feature vectors of 9 and 0, 1, 2, 3 (normals), which means lower ROC-AUC. Lastly, the anomaly score histograms for both cases show similar patterns: the histograms of anomalies are located very close to those of normal data.

The expected reasons for higher performance in the model trained with [0, 1, 2, 3, 4, 5, 6, 7, 8] for normal classes are as follows. When NORMAL_NUM = [0, 1, 2, 3], the model learns only a few kinds of data. So, its ability to distinguish other numbers from them as abnormal can be lowered because it is difficult to distinguish numbers with patterns similar to anomalies from themselves. Since the model has not learned any numbers other than 0, 1, 2, 3, it might not be able to predict other numbers such as 4, 5, 6, 7, 8, and 9 in the test dataset. An abnormal feature (e.g. 9) could form small angles with some of the few normal features, leading to high confidence of misclassification. On the other hand, the ability of the model with more normal classes to distinguish 9 as abnormal will be further improved, and the chances of somewhat parallel alignment are significantly reduced, because the model can learn the difference from 9 better while learning various numbers. Although the anomaly score histograms of both look similar, the histogram of anomalies in more classes has a little bit more portion in the high anomaly score region.

In short, the number of normal classes should be appropriately large enough to make the model work well, and a wider range of NORMAL_NUM in this assignment improved the generalization of the model, resulting in higher AUC-ROC.

[Problem 2]



```

1 # Functions to get Anomaly score for Problem 2
2
3 # cos(theta + m)
4 def cosine_theta_plus_m(cosine, m=MARGIN, easy_margin=False):
5     cos_m = math.cos(m)
6     sin_m = math.sin(m)
7
8     # make the function cos(theta+m) monotonic decreasing while theta in [0°, 180° ]
9     th = math.cos(math.pi - m)
10    mm = math.sin(math.pi - m) * m
11
12    cosine = cosine.clone()
13    cosine = cosine.clip(-1+1e-7, 1-1e-7)
14
15    sine = torch.sqrt(1.0 - torch.pow(cosine, 2))
16    phi = cosine * cos_m - sine * sin_m
17
18    if easy_margin:
19        phi = torch.where(cosine > 0, phi, cosine)
20    else:
21        phi = torch.where((cosine - th) > 0, phi, cosine - mm)
22
23    return phi
24
25 # A = 1 - max_i(p_i)
26 # p0 = cos(theta_0 + m) / {cos(theta_0 + m) + cos(theta_1) + ... + cos(theta_8)}
27 def modified_MSP_score(cosine, scale=SCALE, margin=MARGIN):
28     cosine = cosine.clone()
29     cosine_m = cosine_theta_plus_m(cosine, m=MARGIN, easy_margin=False)
30
31     p_i_list = []
32     for i in range(cosine.shape[1]):
33         logits_i = cosine.clone()
34         logits_i[:, i] = cosine_m[:, i]
35         logits_i *= scale
36         p_i = F.softmax(logits_i, dim=1)[:, i].unsqueeze(1) # p_i.shape = (B, 1)
37         p_i_list.append(p_i)
38
39     p_i_list = torch.cat(p_i_list, dim=1)
40     MSP = torch.max(p_i_list, dim=1).values
41     return torch.tensor(1) - MSP

```

```

# score = MSP_score(SCALE*cosine) # Anomaly Score

# line for Problem 2
score = modified_MSP_score(cosine)

```

To satisfy the conditions, $p_i = \frac{e^{s \cos(\theta_i + m)}}{e^{s \cos(\theta_0)} + \dots + e^{s \cos(\theta_i + m)} + \dots + e^{s \cos(\theta_8)}}$ and $A_2 = 1 - \max_i(p_i)$, of **[Problem 2]**, I modified the function MSP_score implemented in the function 'test'. I

defined two new functions 'cosine_theta_plus_m' which calculate $\cos(\theta + m)$ by the same method in the class 'ArcMarginProduct', and 'modified_MSP_score' to calculate new anomaly score.

For scatter plots, the model with modified anomaly score shows the same plots in 3D and 2D as those of **[Problem 1]** since the only change in **[Problem 2]** is the formula just for anomaly score. Also, it shows lower performance than the model of **[Problem 1]**, with ROC-AUC = 0.846. Unlike the scatter plots and ROC curve, the anomaly score histograms between them considerably differ: the histograms are crowded to both sides near zero and one.

The expected reasons why this happened are as follows. First, the summation of each p_i is not 1: $\sum_i p_i \neq 1$. In normal softmax function, it always ensures that $\sum_i p_i = 1$ where $p_i = \frac{e^{s \cos(\theta_i)}}{\sum_j e^{s \cos(\theta_j)}}$ and it has an effect of normalization. So, we can treat each p_i as probability and can compare among p_i s. However, in new formula for **[Problem 2]**, each p_i looks like a probability, but it is actually incomplete value and distorts the natural comparisons among classes. It produces inconsistent anomaly scores, that is, an anomaly score is very small (almost 0) or very large (almost 1).

Also, since $e^{s \cos(\theta_i + m)}$ is smaller than $e^{s \cos(\theta_i)}$, both numerator and denominator are smaller than the original ones, respectively. The denominator is a sum of 9 terms (from class 0 to class 8), so it might be affected less than the numerator by the reduction. Therefore, it is likely that p_i is smaller than the original one. Then, when all p_i s become quite small, the anomaly score becomes close to 1. This is why some samples have anomaly scores near 1.

From the reasons above, the model with the modified anomaly score has worse performance than the normal one.

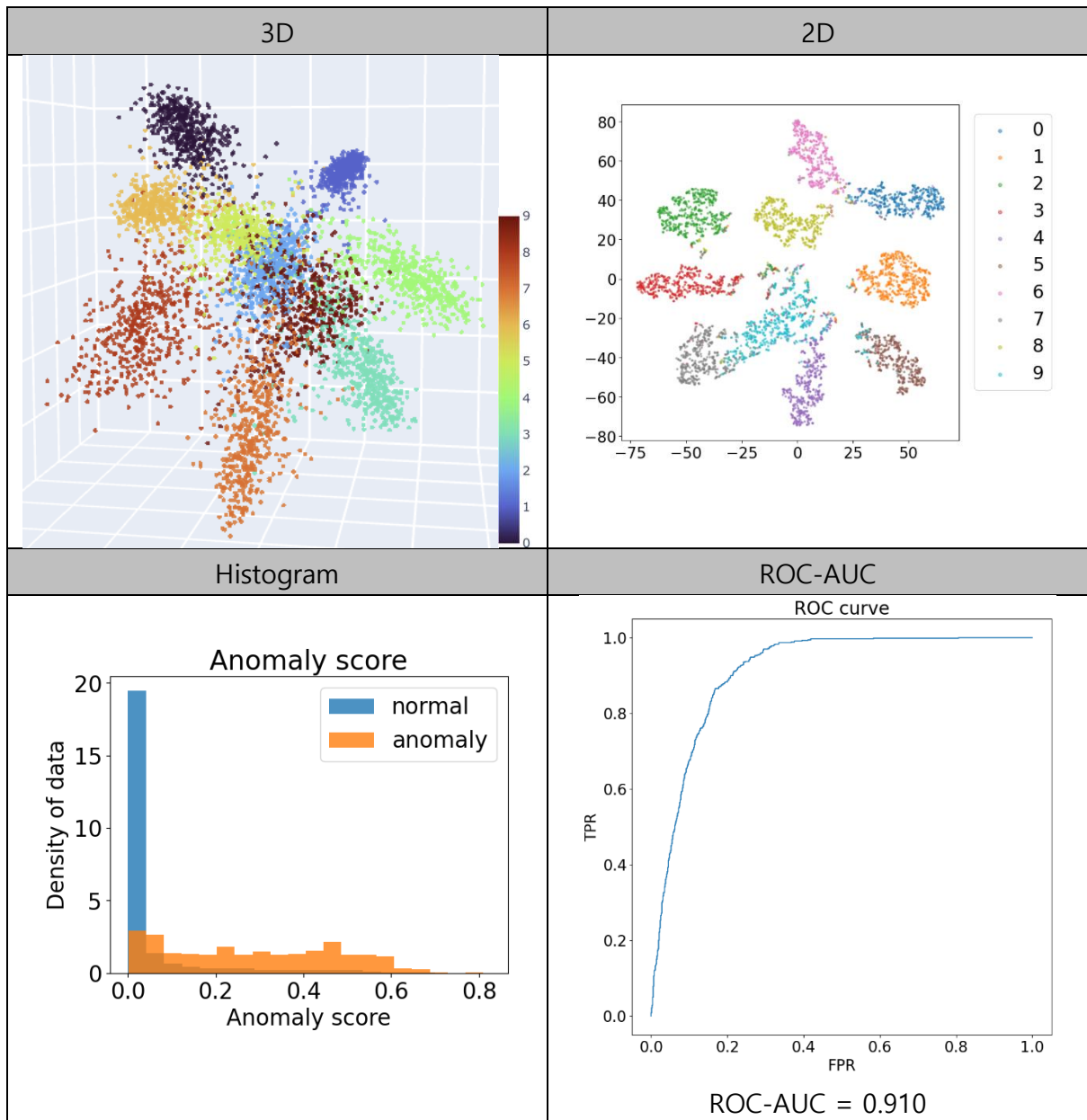
[Problem 3]

```
self.arcface = ArcMarginProduct(in_feature=embsize, out_feature=nclass, s=scale, m=margin, easy_margin=False)
# line for Problem 3
self.fc = nn.Linear(embsize, nclass, bias=False)
```

```
def forward(self, x, p):
    feature = self.feature_extractor(x)
    feature = feature.reshape(feature.shape[0], -1)
    feature = self.features(feature)
    # logits, cosine = self.arcface(feature, p)

    # lines for Problem 3
    logits = self.fc(feature) # dot product:  $w^T z$ 
    cosine = F.linear(F.normalize(feature), F.normalize(self.fc.weight), bias=None)

# line for Problem 3
score = MSP_score(logits)
```



To replace logits l_i in computing loss and anomaly score into $w_i^T z$, I added new linear layer other than 'ArcMarginProduct'. The chart above shows results of the model with plain

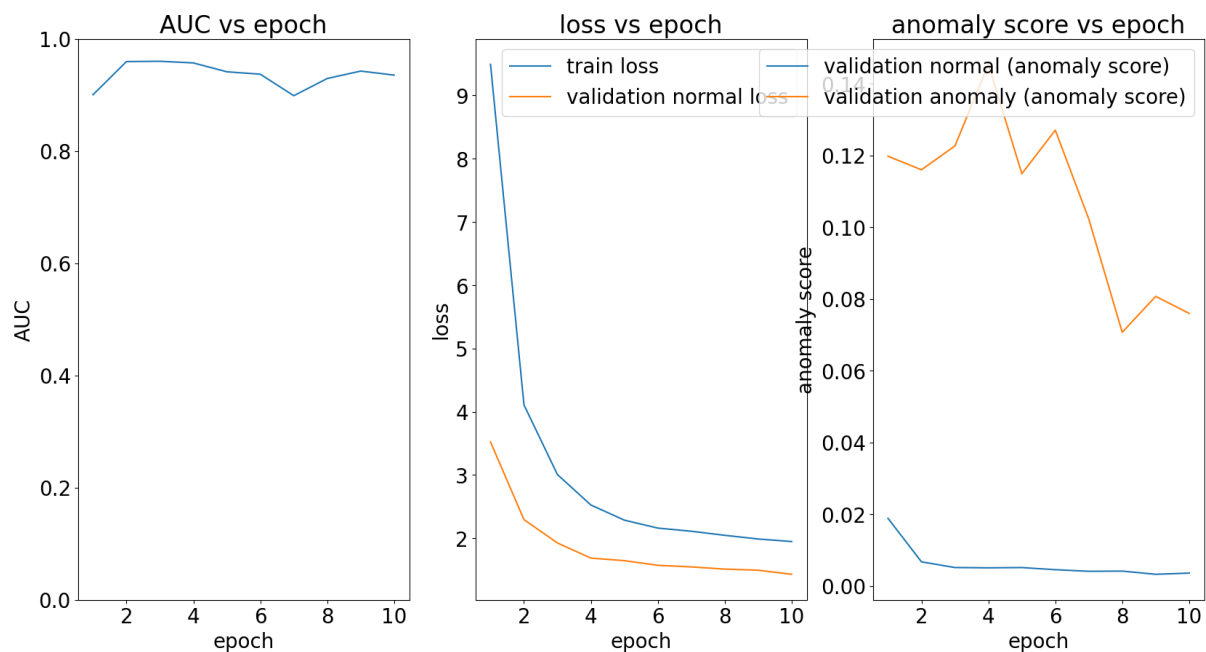
cross-entropy loss.

For scatter plot in 3D, since the feature vectors are not normalized and distributed across huge range, I plotted it in range $[-40, 40]$, not in the original range $[-4, 4]$. The feature vectors are distributed without any directionality, and the distribution of them is not compact and sharp. On the other hand, the features are distributed compactly and clearly along distinct directions in the original one (with ArcFace loss). Similarly, for scatter plot in 2D, each cluster looks like it has no directionality and is less compact compared to the original one. In the histograms, both normal and anomaly histograms are more widely spread out.

In this experiment, I got an unexpected result. Actually, I expected that the performance of the model with plain cross-entropy loss would be lower since ArcFace is a newer technique than the plain CE loss. But it has higher performance (ROC-AUC = 0.910) than the model in **[Problem 1]**. This result might come out because the dataset (MNIST) is quite simple, and especially suitable for the plain CE loss. Also, the increased length of feature vectors might help the model get better anomaly detection in terms of softmax probability values. Therefore, although the clustering is weaker, the overall performance improved in this certain experiment.

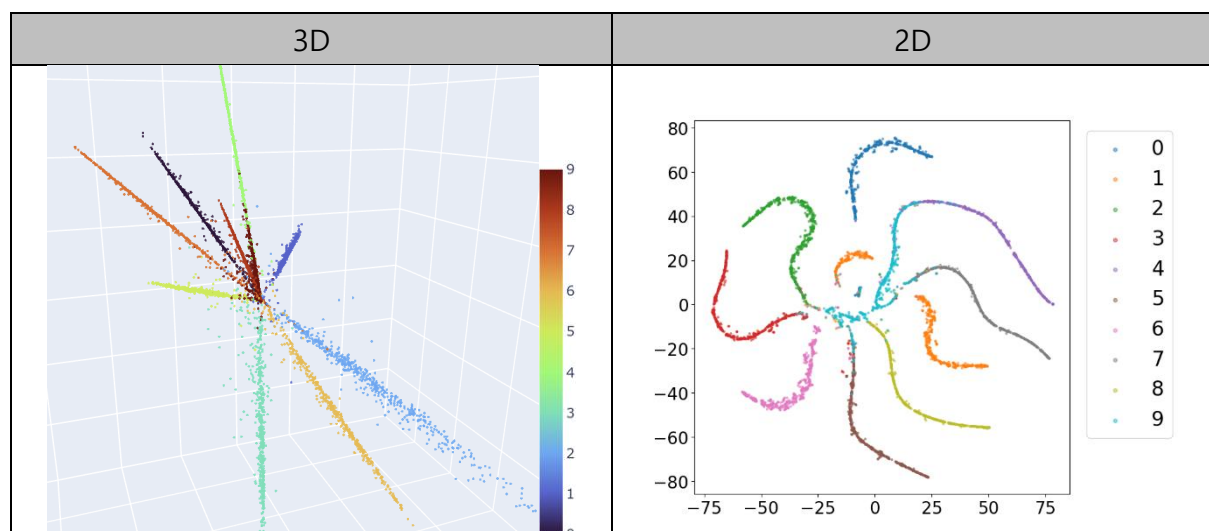
While ArcFace loss makes intra-class compact and inter-class separate well by introducing an angular margin, the plain CE loss does not. With margin, the model falls into a delusion that the value of loss is still very high when it is trained with ground-truth data, so it keeps trying to compact the intra-class feature space and enlarge the inter-class distance. It results in a very narrow and sharp distribution of features in each class. On the other hand, the plain CE loss encourages the model to just enlarge the correct class logits without any geometric constraint (i.e. angle), so that the scatter plot of it in 3D shows the pattern not considering direction of features of each class. Also, since it computes the dot product of not-normalized weight and feature vectors, the value of it varies across the large range. So, the anomaly score histograms are more variably distributed across the range.

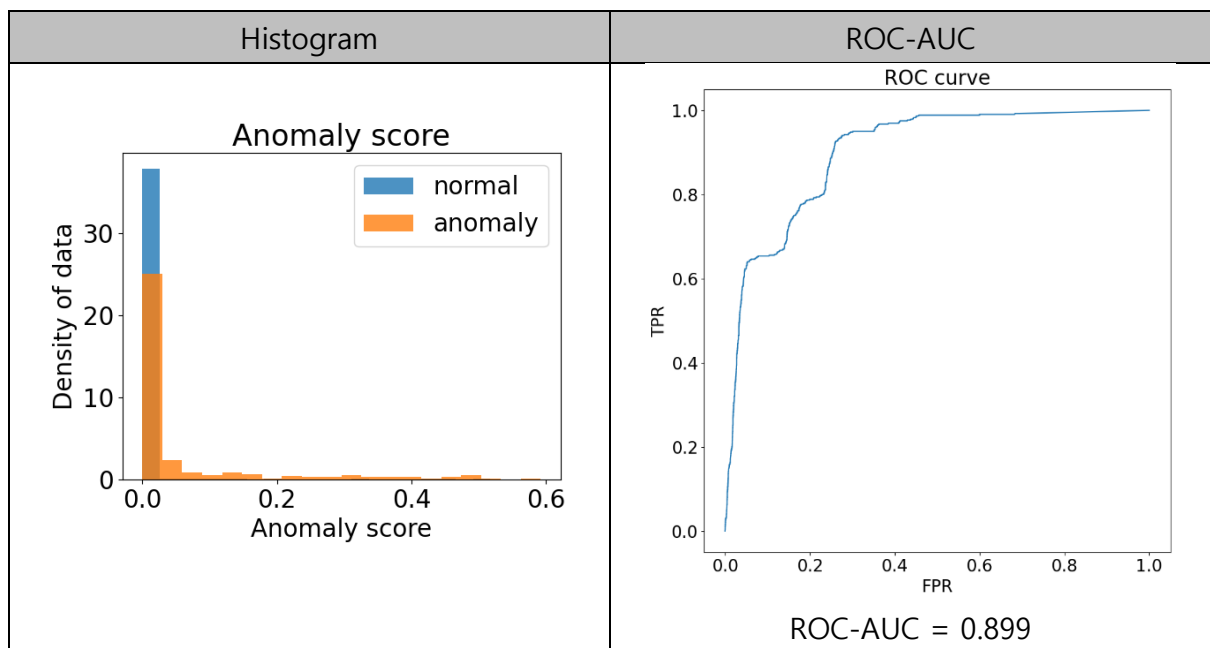
[Problem 4]



One of the basic approaches to improve the performance of the original code is training the model for more epochs. Since the original code trained it only for 5 epochs, I extended epochs to 10 and checked if it is appropriate. The middle graph of the above figure shows the loss vs. epoch (up to 10 epochs). At epoch 5 of the graph, the train and validation losses keep going lower, which means it does not achieve optimal epoch yet. So, I trained it for 10 epochs, and the slopes of those losses at epoch 10 become very small (almost 0).

Also, I changed the hyperparameter 'LBSMOOTH', which controls the label smoothing, from 0 (original) into 0.1 to reduce over confidence of the model and to prevent overfitting. With label smoothing, the model begins to consider not only the ground-truth labels, but also the other labels, so that it predicts an output more carefully.





As a result, the performance of the model improved up to almost 90% in terms of ROC-AUC (0.866 $\rightarrow$ 0.899).